

CMPU4021 Distributed Systems – Labs

Concurrency and Multithreading in Java

Learning Outcomes:

1. Be able to create Java threads using both the Thread class and the Runnable interface.
2. Be able to describe the various states that a Thread can be in, and how to transition between these states.
3. Be able to develop Java code that uses built in locks and monitors.

Tasks

You should read Lab Notes (W3_MultiThreading_LabNotes.pdf and W3_Producer_Consumer_LabNotes.pdf) before you start the tasks; or read it while performing the tasks (as a help).

T1. Examine, compile and execute T1\ThreadShowName.java. Notice how the two threads overlap in execution.

T1.1. Examine, compile and execute T1_1\SleepMessages.java
The SleepMessages example uses sleep to print messages at four-second intervals. Notice that main declares that it throws InterruptedException. This is an exception that sleep throws when another thread interrupts the current thread while sleep is active. Since this application has not defined another thread to cause the interrupt, it doesn't bother to catch InterruptedException.

T2. Examine, compile and execute T2\ThreadHelloCount.java. Notice how the two threads belong to different classes.

T3. Examine, compile and execute T3\RunnableShowName.java and T3\RunnableHelloCount.java. Observe how the Runnable interface is used instead of the Thread class. Observe also, how the instantiated object is still a Thread.

T4. Thread-per-connection approach

In this approach, a single thread is used to handle all of the client's requests. This approach will require that the client send some sort of notification that it has no further requests. In lieu of an explicit notification, a timeout may need to be set to automatically disconnect the client after a sufficient period of time has elapsed.

Examine, compile and execute the server and clients in:

T4\ MultiTheadedServerPerConnection. Java and T4\SimpleClient.java.

T4.1. Examine, compile and execute the client server application (T4_1\MultiEchoServer.java and T4_1\MultiEchoClient.java).

T4.2. Examine, compile and execute T4_2\SimpleThreads.java
SimpleThreads consists of two threads. The first is the main thread that every Java application has. The main thread creates a new thread from the Runnable object, MessageLoop, and waits for it to finish. If the MessageLoop thread takes too long to finish, the main thread interrupts it.

The MessageLoop thread prints out a series of messages. If interrupted before it has printed all its messages, the MessageLoop thread prints a message and exits.

T5. Examine, compile and execute T5\SemApp.java.

In some enterprise systems, it's not uncommon for developers to need to throttle the number of open requests (threads/actions) against a particular resource. While it is possible to try to write the throttling code by hand, it is easier to use the semaphore class, which takes care of throttling for you, as shown in SemApp.java:

Even though the 10 threads in this sample are running, only three are active. The other seven are held at bay until one of the semaphore counts is released. The Semaphore class supports acquiring and releasing more than one *permit* at a time, but it would not be appropriate in this scenario.

Note: The concept of “throttling concurrency,” refers to the number of threads that are allowed to do work at a particular time. It is a policy to decide how many threads can be run simultaneously without hurting performance. It is a way to limit the amount of possible active threads at one time.

T6. Thread pools

Thread pools are useful when the number of threads that are created need to be limited. Using a pool not only controls how many threads are created, but it can also eliminate the need to create and destroy threads repeatedly, an often expensive operation. Requests are assigned to threads in the pool. Some thread pools will create new threads if there are no unused threads available. Others will restrict the number of threads available. This may result in some requests being blocked.

Thread pools can be implemented using the `ThreadPoolExecutor` class. This class also provides methods that deliver status information regarding thread execution.

The `ThreadPoolExecutor` class possesses several constructors, but the `Executors` class provides an easy way of creating instances of the `ThreadPoolExecutor` class. We will demonstrate two of these methods. First, we will use the `newCachedThreadPool` method. The pool created by this method will reuse threads. New threads will be created when needed. However, this can result in too many threads being created. The second method, `newFixedThreadPool`, creates a fixed-size pool of threads.

Examine, compile and run the code and documentation in the T6 directory.

The server will return a price of a part when given the name of a part. Each thread will access a `ConcurrentHashMap` instance that holds the part information. We use the concurrent version of the hash map as it may be accessed from multiple threads.

`SimpleClient.java` uses the `Socket` class to establish a connection to the server. Input and output streams are used to send and receive responses.

Server:

```
➤ java -classpath . ThreadPool
```

Client:

```
➤ java -classpath . SimpleClient
```

T7: You may work on this task at home.

Write a server that maintains a list of **multiple** clients. Clients take turns sending messages to the server. When the server receives a message from either client it sends the received message to the other client. Use multi-threading where appropriate.

T8: Examine, compile and run the code and documentation in the

`Producer_Consumer_Problem` directories. Read `W3_Producer_Consumer_LabNotes.pdf` file to understand different versions of the code.

To run it, call Main class:

```
➤ java Main
```

T9. Download `T9\ResourceServer.java`, `T9\Resource.java`, `T9\ClientThread.java` and `T9\Producer.java`. Take a look, compile and run the server. Observe the operation of the server and the synchronized blocks.

Note: This is a producer-consumer problem, in which a producer is generating instances of some resource and a consumer is removing instances of the resource. The resource is modelled by the *Resource* class, the producer by the *Producer* class. The *Producer* class is a thread class, extending class *Thread*. The server program, *ResourceServer* creates *Resource* object and then a *Producer* thread, passing the constructor for this thread a reference to the *Resource* object. The server then starts the thread running and begin accepting connections from client. As each client makes connection, the server creates an instance of *ClientThread* (another *Thread* class), which is responsible for handling all subsequent dialogue with the client.

T10. Write a client for the *ResourceServer*. Call it *ConsumerClient*.

Note: Take a copy of *MultiEchoClient*, renaming it *ConsumerClient*. Using this file as a template, modify the code that it acts as a client to *ResourceServer*. Ensure that the user can pass only 0 or 1 to the server (e.g. show an error if user enters 2). Test the operation of the server with two clients.

Further Reading

Java Tutorial section on Concurrency

<https://docs.oracle.com/javase/tutorial/essential/concurrency/>

<https://docs.oracle.com/javase/tutorial/essential/concurrency/highlevel.html>

<https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/lang/Thread.html>

Chapter 7, Learning Network Programming with Java, Richard M. Reese, Packt, 2015